

Oblig 4 – NoSQL

Installasjon

Først startet jeg med å installere MongoDB etter oppgavens instruksjoner. Etter dette valgte jeg å opprette en privat konto på MongoDB domenet slik at jeg kunne opprette en gratis Atlas server. I tillegg til dette har jeg installert «mongo shell» slik at jeg kan kjøre spørringer og script direkte fra «command prompt» terminalen.

Import

Først startet jeg med å legge til en «_id» kolonne i import filene, slik at jeg fikk en egendefinert objekt id på dokumentene. Deretter importere jeg alle CSV filene fra tidligere obliger rett inn i NoSQL databasen. Dette fungerte greit, men siden det ikke er like lett å kjøre JOINT spørringer mellom NoSQL kolleksjoner fremfor slik det er med relasjonsdatabaser i en SQL servere så valgte jeg å endre noe på datastrukturen.

Når dette var utført, hadde jeg fire kolleksjoner i databasen med navnene;

- Pasienter
- Innleggelser
- Leger
- Provins

Disse hadde følgende struktur i sine dokumenter.

Pasienter	Innleggelser	Leger	Provins
<pre>{ "_id": "p1", "pasient_id": 1, "fornavn": "Donald", "etternavn": "Waterfield", "kjonn": "M", "fodselsdag": "12.02.1963", "hoyde": 156, "vekt": 65, "sted": "Barrie", "provins_id": "NO", "allergier": null, }</pre>	<pre>{ "_id": "i1", "pasient_id": 1, "innleggesdato": "06.11.2018", "utskrivningsdato": "08.11.2018", "diagnose": "Ovarian Dermoid-Cyct", "lege_id": 21 }</pre>	<pre>{ "_id": "l1", "lege_id": 1, "fornavn": "Claude", "etternavn": "Walls", "spesialitet": "Internist" }</pre>	<pre>{ "_id": "pr1", "provins_id": "AB", "provins_navn": "Alberta" }</pre>

Struktur

Dette er en løsning som ville fungert fint i en relasjonsdatabase, men i NoSQL vil det bli tungvint og holde dette systemet oppdatert, da man ikke får knyttet forskjellige kolleksjoner direkte mot hverandre. Eksempelvis å knytte et pasient dokument opp mot tilhørende innlegges dokumenter med felles pasient_id. Dette valgte jeg derfor å løse med å opprette Array lister i pasienter dokumentene som holdt på pasientens tilhørende innleggelse _id i pasient

dokumentet. Jeg løste dette ved å hente ut dokument id fra innleggelser kolleksjonen og pusset disse inn i innleggelser Array listen tilhørende dokumenter i pasienter kolleksjonen. For å finne frem til denne løsningen har jeg brukt flere kilder som ChatGPT(OpenAI, 2020), youtube(Net Ninja, 2022)(Bro Code, 2023) og pensumbok(Kristoffersen, 2021, kap. 15). Dette er JS scriptet jeg kjørte i mongosh for å utføre handlingen:

innleggelserToPasienter.js
<pre> db.pasienter.aggregate([{ \$lookup: { from: "innleggelser", localField: "pasient_id", foreignField: "pasient_id", as: "innleggelser" } }, { \$unwind: { path: "\$innleggelser", preserveNullAndEmptyArrays: true } }, { \$group: { _id: "\$_id", pasient_id: { \$first: "\$pasient_id" }, fornavn: { \$first: "\$fornavn" }, etternavn: { \$first: "\$etternavn" }, kjonn: { \$first: "\$kjonn" }, fødselsdag: { \$first: "\$fødselsdag" }, høyde: { \$first: "\$høyde" }, vekt: { \$first: "\$vekt" }, sted: { \$first: "\$sted" }, provins: { \$first: "\$provins" }, allergier: { \$first: "\$allergier" }, innleggelser: { \$push: "\$innleggelser._id" } } }, { \$merge: { into: "pasienter", whenMatched: "replace" } }]) </pre>

Med tanke på hvordan jeg nå hadde strukturert objekt id på alle dokumentene kunne jeg nå bytte ut «pasient_id» og «lege_id» feltene i innleggelser dokumentene. Jeg valgte å gjøre dette siden _id allerede er indeksert i sine kolleksjoner, og ved å bytte ut disse ville jeg få effektive søkemuligheter uten å måtte opprette unødvendige indekser for dette. Jeg løste dette ved å kjøre følgende script mot innleggelser kolleksjonen:

legeldInnleggelser.js

```
db.innleggelser.aggregate([
  {
    $lookup: {
      from: "leger",
      localField: "lege_id",
      foreignField: "lege_id",
      as: "legeInfo"
    }
  },
  {
    $unwind: "$legeInfo"
  },
  {
    $project: {
      _id: 1,
      pasient_id: 1,
      innleggelsesdato: 1,
      utskrivningsdato: 1,
      diagnose: 1,
      lege: "$legeInfo._id"
    }
  },
  {
    $out: "innleggelser"
  }
])
```

pasientToInnleggelser.js

```
db.innleggelser.aggregate([
  {
    $lookup: {
      from: "pasienter",
      localField: "pasient_id",
      foreignField: "pasient_id",
      as: "pasientInfo"
    }
  },
  {
    $unwind: "$pasientInfo"
  },
  {
    $project: {
      _id: 1,
      pasient: "$pasientInfo._id",
      innleggelsesdato: 1,
      utskrivningsdato: 1,
      diagnose: 1,
      lege: 1
    }
  },
  {
    $out: "innleggelser"
  }
])
```

Når dette var utført kontrollerte jeg at `_id` referanser var på riktig sted, før jeg fjernet feltene «`pasient_id`» og «`lege_id`» fra dokumentene i kolleksjonene «`leger`» og «`pasienter`». Disse feltene var ikke lenger nødvendige siden `_id` nå holder på disse verdiene og er ferdig indeksert. Jeg fjernet disse feltene ved å gjøre spørringer direkte i mongosh.

```
Atlas atlas-x9sgvh-shard-0 [primary] oblig4> db.pasienter.updateMany( {}, { $unset: { "pasient_id": "" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 500,
  modifiedCount: 500,
  upsertedCount: 0
}
Atlas atlas-x9sgvh-shard-0 [primary] oblig4> db.leger.updateMany( {}, { $unset: { "lege_id": "" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 27,
  modifiedCount: 27,
  upsertedCount: 0
}
```

Til slutt gjorde jeg en tilsvarende endring i «`provins_id`» feltet tilhørende dokumentene i «`pasienter`» kolleksjonen. Dette var ikke noe som var av stor nødvendighet, men for å holde strukturen konsis og siden `_id` i «`provins`» kolleksjonen og allerede er indeksert, så tenkte jeg at dette kunne være greit å få gjort. Jeg gjorde endringen med følgende script:

provinsToPasienter.js

```
db.pasienter.aggregate([
  {
    $lookup: {
      from: "provins",
      localField: "provins",
      foreignField: "provins._id",
      as: "provinsInfo"
    }
  },
  {
    $unwind: "$provinsInfo"
  },
  {
    $project: {
      _id: 1,
      pasient_id: 1,
      fornavn: 1,
      etternavn: 1,
      kjonn: 1,
      fodselsdag: 1,
      hoyde: 1,
      vekt: 1,
      sted: 1,
      provins: "$provinsInfo._id",
      allergier: 1,
      innleggelser: 1
    }
  },
  {
    $out: "pasienter"
  }
])
```

På denne måten endte jeg opp med fire kolleksjoner som holder på den samme informasjonen og refererer til de samme dokumentene på tvers av kolleksjoner, som er ferdig indeksert basert på `_id`. Dette er slik dokumentstrukturen så ut i de forskjellige kolleksjonene etter at endringer var utført.

Pasienter	Innleggelser	Leger	Provins
<pre>{ "_id": "p1", "fornavn": "Donald", "etternavn": "Waterfield", "kjonn": "M", "fodselsdag": "12.02.1963", "hoyde": 156, "vekt": 65, "sted": "Barrie", "allergier": null, "innleggelser": ["i1", "i2"], "provins": "pr9" }</pre>	<pre>{ "_id": "i1", "innleggelsesdato": "06.11.2018", "utskrivningsdato": "08.11.2018", "diagnose": "Ovarian Dermoid-Cyct", "lege": "l21", "pasient": "p1" }</pre>	<pre>{ "_id": "l1", "fornavn": "Claude", "etternavn": "Walls", "spesialitet": "Internist" }</pre>	<pre>{ "_id": "pr1", "provins_id": "AB", "provins_navn": "Alberta" }</pre>

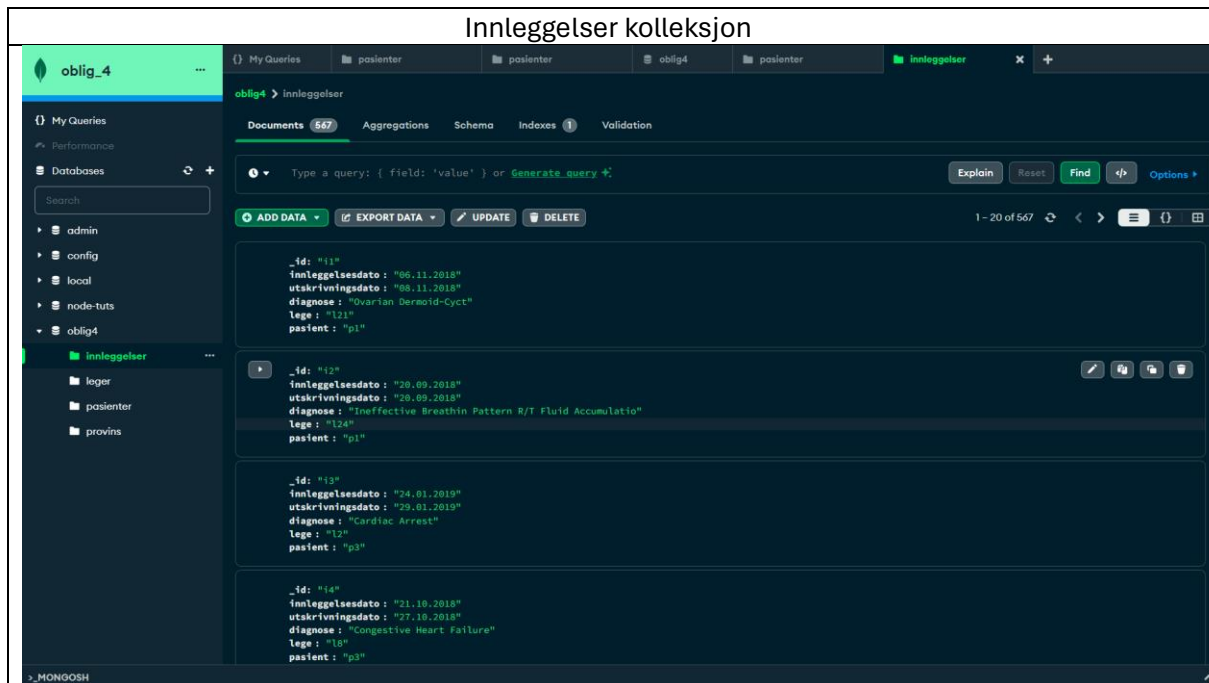
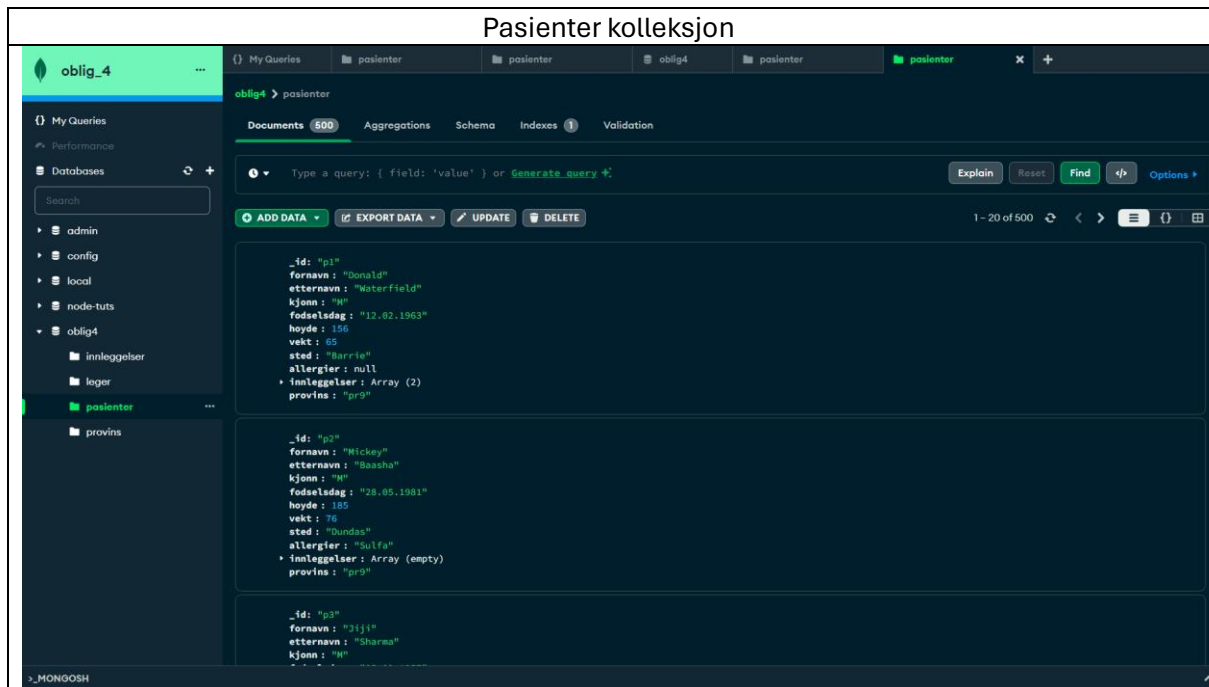
Indeksering

Til slutt indekserte jeg «innleggelser» kolleksjonen etter «diagnoser» feltet og «pasienter» kolleksjonen etter «etternavn» feltet. Dette ble kun gjort manuell indeksering i «pasienter» og «innleggelser» kolleksjonene, da de andre kolleksjonene ikke holder på betydelige store mengder med data og vil generelt bli lite søkt i forhold til disse kolleksjonene. Det vil jo også være en mulig vurdering om pasienter skal indekseres etter fornavn, eller innleggelser etter diagnoser eller dato. Alt ettersom hvilke spørringer som ville blitt mest foretatt mot serveren. Indekseringene ble opprettet direkte via moshos:

```
Atlas atlas-x9sgvh-shard-0 [primary] oblig4> db.innleggelser.createIndex({"diagnose":1})
diagnose_1
Atlas atlas-x9sgvh-shard-0 [primary] oblig4> db.pasienter.createIndex({"etternavn":1})
etternavn_1
Atlas atlas-x9sgvh-shard-0 [primary] oblig4>
```

Bilder

Her en noen bilder av hvordan den endelige basen ble sende ut i MongoDB Compass:



Databasesystemer – Oblig 4 – Andreas Isaksen

Leger kolleksjon

oblig4 > leger

Documents 27 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

[ADD DATA](#) [EXPORT DATA](#) [UPDATE](#) [DELETE](#) 1 - 20 of 27

_id	fornavn	etternavn	spesialitet
"11"	"Claude"	"Walla"	"Internist"
"12"	"Joshua"	"Green"	"Cardiologist"
"13"	"Miriam"	"Fregre"	"General Surgeon"
"14"	"James"	"Russo"	"Obstetrician/Gynecologist"
"15"	"Scott"	"Hill"	"Gastroenterologist"

> MONGODB

Provins kolleksjon

oblig4 > provins

Documents 13 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

[ADD DATA](#) [EXPORT DATA](#) [UPDATE](#) [DELETE](#) 1 - 13 of 13

_id	provins_id	provins_navn
"pr1"	"AB"	"Alberta"
"pr2"	"BC"	"British Columbia"
"pr3"	"MB"	"Manitoba"
"pr4"	"NB"	"New Brunswick"
"pr5"	"NL"	"Newfoundland and Labrador"
"pr6"	"NT"	"Northwest Territories"

> MONGODB

Litteraturliste

OpenAI. (2020). GPT-3.5 [Stor språkmodell]. <https://chat.openai.com/share/b8df98c9-35eb-4ed8-a149-4af7ac41418d>

Net Ninja. (2022, 21.mars). Complete MongoDB Tutorial [Videospilleliste]. YouTube. https://www.youtube.com/watch?v=ExcRbA7fy_A&list=PL4cUxeGkcC9h77dJ-QJlwGIZITd4ecZOA

Bro Code. (2023, 14.april). Learn MongoDB in 1 Hour [Video]. YouTube. <https://www.youtube.com/watch?v=c2M-rlkkT5o&t=837s>

Kristoffersen, B. (2021). Databasesystemer (2. Oppslag). Universitetsforlaget. ISBN 978-82-15-03251-1